



POZNAN UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTING AND TELECOMMUNICATION
Institute of Computing Science

Master's thesis

**LATICE BASED POST-QUANTUM CRYPTOGRAPHY:
IMPLEMENTING AND ANALYZING KYBER ALGORITHM, NIST
POST-QUANTUM CRYPTOGRAPHY FINALIST**

Konrad Kłodziński, 145234

Supervisor
dr. inż. Anna Grocholewska-Czuryło

POZNAŃ 2024

Abstract

English

This thesis presents a comprehensive study on lattice-based post-quantum cryptography, focusing specifically on the Kyber algorithm, a finalist in the NIST Post-Quantum Cryptography Standardization Project. The motivation behind this research stems from the growing threat posed by quantum computers to classical cryptographic systems. Quantum computers, leveraging principles of quantum mechanics, can potentially break widely used cryptographic protocols, necessitating the development of quantum-resistant algorithms.

The Kyber algorithm is a lattice-based Key Encapsulation Mechanism (KEM) that provides strong security guarantees based on the hardness of problems related to lattices, such as the Learning With Errors (LWE) problem. This research includes a detailed implementation of the Kyber algorithm using modern C++ standards and tools. Performance benchmarks were conducted using Intel VTune Profiler on a Windows 11 system, demonstrating the algorithm's efficiency in key generation, encapsulation, and decapsulation processes.

Key contributions of this thesis include an in-depth analysis of the Kyber algorithm's mathematical foundations, implementation details, performance evaluation, and security considerations. The findings highlight the robustness of Kyber against quantum attacks and its applicability in real-world cryptographic applications. Furthermore, the implementation insights provided in this thesis offer valuable guidance for future developments in post-quantum cryptography.

This work concludes with a summary of the algorithm's performance and security features, discussing potential areas for further research and optimization. The results contribute to the ongoing efforts in securing digital communications against the advent of quantum computing technologies.

Polski

Niniejsza praca przedstawia kompleksowe badanie kryptografii postkwantowej opartej na kratkach, koncentrując się na algorytmie Kyber, który jest finalistą projektu standaryzacji kryptografii postkwantowej NIST. Motywacja do przeprowadzenia tych badań wynika z rosnącego zagrożenia, jakie komputery kwantowe stanowią dla klasycznych systemów kryptograficznych. Komputery kwantowe, wykorzystując zasady mechaniki kwantowej, mogą potencjalnie złamać powszechnie stosowane protokoły kryptograficzne, co wymaga opracowania algorytmów odpornych na ataki kwantowe.

Algorytm Kyber jest mechanizmem kapsułkowania kluczy (KEM) opartym na kratkach, który zapewnia silne gwarancje bezpieczeństwa oparte na trudności rozwiązywania problemów związanych z kratami, takich jak problem uczenia się z błędami (LWE). Badania te obejmują szczegółową implementację algorytmu Kyber z wykorzystaniem nowoczesnych standardów i narzędzi C++. Przeprowadzono testy wydajności przy użyciu narzędzia Intel VTune Profiler na systemie Windows 11, wykazując efektywność algorytmu w procesach generowania kluczy, kapsułkowania i dekapsulacji.

Kluczowe wkłady tej pracy obejmują dogłębną analizę matematycznych podstaw algorytmu Kyber, szczegóły implementacji, ocenę wydajności i rozważania dotyczące bezpieczeństwa. Wyniki podkreślają odporność algorytmu Kyber na ataki kwantowe i jego zastosowanie w rzeczywistych aplikacjach kryptograficznych. Ponadto, wnioski z implementacji przedstawione w tej pracy oferują cenne wskazówki dla przyszłych badań i rozwoju w dziedzinie kryptografii postkwantowej.

Praca kończy się podsumowaniem wydajności i cech bezpieczeństwa algorytmu, omawiając potencjalne obszary do dalszych badań i optymalizacji. Wyniki przyczyniają się do trwających wysiłków w zabezpieczaniu komunikacji cyfrowej przed nadejściem technologii komputerów kwantowych.

Tutaj będzie karta pracy dyplomowej;
oryginał wstawiamy do wersji dla archiwum PP, w pozostałych kopiach wstawiamy ksero.

Contents

1	Introduction	1
1.1	Background	1
1.2	Motivation	1
1.3	Objective	1
1.4	Structure of the Thesis	1
2	Quantum Computing	3
2.1	Introduction	3
2.2	Scientific Background	3
2.3	Difference Between Classical and Quantum Computers	4
2.3.1	Quantum Gates Acting on One Qubit	4
2.3.2	Qubit States	5
2.3.3	Measurement in Multi-Qubit Systems	5
2.3.4	Quantum Entanglement	6
	Measuring the Bell State	6
2.3.5	Advantages of Quantum Entanglement	6
	CNOT Operator	7
2.3.6	Quantum Teleportation	7
2.4	BB84 Protocol	8
3	Lattice-based Cryptography	9
3.1	Lattices	9
3.2	Problems Involving Lattices	10
3.2.1	Shortest Vector Problem (SVP)	10
3.2.2	Closest Vector Problem (CVP)	10
3.2.3	Lenstra–Lenstra–Lovász (LLL) Algorithm	10
3.3	GGH Encryption Scheme	11
3.3.1	Algorithm Overview	11
	Key Generation	11
	Encryption	11
	Decryption	12
3.3.2	Pseudo Code	12
3.3.3	Security Considerations	12
3.4	Learning with and without errors	12
3.4.1	Learning without errors	12
3.4.2	Learning with errors	13
3.4.3	Encoding messages with LWE	13

3.4.4	Re-framing the Learning with Errors problem as a Lattice problem	14
4	Kyber Algorithm	15
4.1	Introduction	15
4.2	Mathematical Foundations	15
4.2.1	Learning With Errors (LWE) Problem	15
	Mathematical Formulation of LWE	15
4.2.2	Module Learning With Errors (MLWE) Problem	16
	Mathematical Formulation of MLWE	16
4.2.3	Detailed Explanation of the MLWE Problem	16
	Matrix $\mathbf{A}$	16
	Secret and Error Vectors	16
	Noise Addition	17
	Problem Hardness	17
4.3	Kyber Algorithm Components	17
4.3.1	Key Generation	17
	Explanation of Key Generation	17
4.3.2	Encapsulation	17
	Explanation of Encapsulation	18
4.3.3	Decapsulation	18
	Explanation of Decapsulation	18
4.4	Security Considerations	19
4.4.1	Security Properties	19
4.4.2	Resistance to Quantum Attacks	19
4.4.3	Error Vectors and Noise	19
4.5	Significance and Applications	19
5	Implementation	20
5.1	Project Structure	21
5.1.1	Performance	22
	RSA Encryption	22
	KEM Encryption	23
5.1.2	API Description	24
6	Conclusions	25
6.1	Summary of Findings	25
6.2	Performance Analysis	25
6.3	Security Considerations	25
6.4	Implementation Insights	25
6.5	Future Work	26
6.6	Final Remarks	26
	Bibliography	27

Chapter 1

Introduction

1.1 Background

The rapid advancement of quantum computing poses a significant threat to traditional cryptographic systems. Quantum computers, leveraging the principles of superposition and entanglement, have the potential to solve complex mathematical problems exponentially faster than classical computers. This capability directly impacts the security of widely-used cryptographic algorithms such as RSA and ECC, which rely on the computational difficulty of problems like integer factorization and elliptic curve discrete logarithms. With the advent of quantum computing, these problems could be solved efficiently, rendering current cryptographic methods vulnerable.

1.2 Motivation

In response to the looming threat posed by quantum computers, the field of post-quantum cryptography has emerged. This field aims to develop cryptographic algorithms that are secure against attacks by both classical and quantum computers. Among the various approaches, lattice-based cryptography has gained significant attention due to its strong security guarantees and efficiency. The National Institute of Standards and Technology (NIST) has been at the forefront of standardizing post-quantum cryptographic algorithms through its Post-Quantum Cryptography Standardization Project. One of the finalists in this project is the Kyber algorithm, a lattice-based Key Encapsulation Mechanism (KEM).

1.3 Objective

This thesis aims to implement and analyze the Kyber algorithm, providing a comprehensive study of its inner workings, performance, and security features. The Kyber algorithm is based on the Learning With Errors (LWE) problem, a hard problem in lattice-based cryptography that offers robust security guarantees. By implementing Kyber, this thesis seeks to contribute to the understanding and practical application of post-quantum cryptographic algorithms.

1.4 Structure of the Thesis

The first chapter provides an overview of the background, motivation, and objectives of the thesis. It introduces the challenge posed by quantum computing to classical cryptographic systems and underscores the necessity for developing post-quantum cryptographic solutions.

Chapter 2 delves into the fundamentals of quantum computing, explaining the scientific background and the differences between classical and quantum computers. Key concepts such as quantum gates,

qubit states, quantum entanglement, and teleportation are covered. The chapter also discusses the BB84 protocol, a quantum key distribution scheme, and its relevance to secure communication.

Chapter 3 introduces lattice-based cryptography, focusing on its mathematical foundations and the hard problems that underpin its security, such as the Shortest Vector Problem (SVP) and the Closest Vector Problem (CVP). It explores the GGH encryption scheme, which leverages the hardness of finding short vectors in a lattice for encryption. The security of GGH relies on the difficulty of solving these lattice problems, making it a robust method for cryptography. The chapter also covers the Learning With Errors (LWE) problem, a cornerstone of modern lattice-based cryptography. The LWE problem involves solving linear equations perturbed by small random errors, providing a strong foundation for cryptographic schemes.

Chapter 4 offers a comprehensive examination of the Kyber algorithm, a lattice-based Key Encapsulation Mechanism (KEM) that stands out as a leading candidate for post-quantum cryptography. The chapter delves into the mathematical foundations of the algorithm, focusing on the Module Learning With Errors (MLWE) problem, and provides detailed descriptions of the key generation, encapsulation, and decapsulation processes. It also highlights the security features derived from the hardness of lattice problems and discusses the algorithm's robustness against quantum attacks. Additionally, the chapter underscores the significance and potential applications of Kyber in securing future communications.

Chapter 5 describes the implementation of the Kyber algorithm using C++²³. It covers the project structure, development environment, and tools used, including CMake for build management and Docker for consistent development and testing environments. The chapter details the integration of OpenSSL for cryptographic functions and the performance benchmarks of the implemented Kyber algorithm, comparing it with traditional RSA encryption. It provides a detailed API description, highlighting the simplicity and usability of the Kyber API compared to the RSA API.

The final chapter summarizes the findings of the thesis, discussing the implications of the results and potential future work. It reflects on the importance of post-quantum cryptography in securing digital communication in the quantum era and the contributions made by this thesis to the field.

Chapter 2

Quantum Computing

2.1 Introduction

Based on NSA's FAQ [1] about Quantum Computers, a sufficient Cryptographically Relevant Quantum Computer (CRQC) would be capable of compromising public key and digital signature algorithms, thus exposing classified and critical information. Despite the fact that, if possible, sufficient CRQCs are years away, there exists a phenomenon of Harvest Now, Decrypt Later (HNDL) [2]. HNDL is an intelligence strategy based on acquiring encrypted data in hopes of decrypting it later when consumer quantum computers are available. These threats are the driving force of today's post-quantum cryptography.

2.2 Scientific Background

The groundwork for Quantum Computing can be traced back to the 1960s and the invention of conjugate coding by Stephen Wiesner. Wiesner's method created a system of transmitting messages where intercepting one destroys the others. The same idea of resistance to compromise would later be used in the BB84 algorithm described later in the chapter.

In the 1980s, three seminal papers by Paul Benioff, Richard Feynman, and David Deutsch demonstrated that a computing system based on the quantum mechanics of particles in superposition is possible. In his 1980 paper, Benioff [3] described a Turing machine based on Schrödinger's equation. In 1981, Feynman gave a talk titled "Simulating Physics with Computers" [4] in which he discussed the potential of simulating quantum phenomena using quantum computers. These computers operate using the principles of superposition and entanglement, fundamental aspects of quantum mechanics, allowing them to perform calculations at speeds unattainable by classical computers. In his 1985 paper [5], David Deutsch expanded on the limitations of classical computing models in simulating quantum systems and argued for the necessity of quantum computers to accurately model quantum phenomena. He extended the Church-Turing principle, traditionally associated with classical computing, to the realm of quantum physics, suggesting that every physically realizable system can be perfectly simulated by a universal quantum computer.

A significant advancement in quantum computing came from Peter W. Shor's paper titled "Algorithms for Quantum Computation: Discrete Logarithms and Factoring." Shor provided a method for factorizing integers on a quantum computer, demonstrating that with an adequate quantum computer, it is possible to break asymmetric cryptography keys. Asymmetric cryptography, commonly used in digital security, relies on pairs of keys—a public key and a private key. The security of these cryptographic systems is based on the computational difficulty of factoring large numbers, a task that classical computers find extremely challenging. Shor's algorithm showed that a sufficiently powerful quantum computer could perform this factorization quickly and efficiently.

Currently, quantum systems are approximately 100 qubits in size, which is significantly less than the estimated 10^7 qubits required to achieve a Cryptographically Relevant Quantum Computer. Despite these limitations, Google achieved "Quantum Supremacy" in October 2019, demonstrating that a quantum computer could complete a task in 200 seconds that would take a classical supercomputer 5 days to perform, instead of the originally claimed 10,000 years.

2.3 Difference Between Classical and Quantum Computers

Bit (Classical)	Qubit [6] (Quantum)
Deterministic States 0 or 1	Superposition States A linear combination of states, represented as $\alpha 0\rangle + \beta 1\rangle$, where $\alpha, \beta \in \mathbb{C}$ and $ \alpha ^2 + \beta ^2 = 1$
Logical Operations Operations such as AND, OR, NOT	Quantum Operations Operations are performed by unitary transformations (quantum gates) like Hadamard, Pauli-X, Y, Z, and controlled operations like CNOT
State Measurement State is directly observable without change	Probabilistic Measurement Measurement collapses the qubit into one of the basis states $ 0\rangle$ or $ 1\rangle$, with probabilities $ \alpha ^2$ and $ \beta ^2$ respectively
Information Dupli- cation Bits can be copied without limitation	No-cloning Theorem Qubits cannot be perfectly copied due to the no-cloning theorem

TABLE 2.1: Comparative overview of classical bits and quantum bits (qubits).

The output of each qubit is determined by a linear combination of states $\alpha|0\rangle + \beta|1\rangle$ where the output is determined as 0 with a probability of $|\alpha|^2$ and 1 with a probability of $|\beta|^2$. Since the output is not deterministic, almost all quantum algorithms are probabilistic, sampling the probability distribution determined by the system's gates.

2.3.1 Quantum Gates Acting on One Qubit

$$a|0\rangle + b|1\rangle \xrightarrow{\boxed{X}} a|1\rangle + b|0\rangle$$

FIGURE 2.1: X gate

$$a|0\rangle + b|1\rangle \xrightarrow{\boxed{Z}} a|1\rangle - b|0\rangle$$

FIGURE 2.2: Z gate

$$a|0\rangle + b|1\rangle \xrightarrow{\boxed{H}} \frac{a}{\sqrt{2}}(|0\rangle + |1\rangle) + \frac{b}{\sqrt{2}}(|0\rangle - |1\rangle)$$

FIGURE 2.3: Hadamard gate

Quantum gates are fundamental components of quantum computers, analogous to classical logic gates used for logical operations. The X gate (Figure 2.1), also known as the quantum NOT gate, reverses the states of a qubit, functioning similarly to a classical NOT gate.

The Z gate (Figure 2.2), or phase gate, changes the phase of a qubit. For a qubit in the phase state $|1\rangle$, the Z gate changes the sign of this state while leaving the state $|0\rangle$ unchanged. This operation modifies the superposition of quantum states, which is crucial for many quantum algorithms.

The Hadamard gate (Figure 2.3), also known as the H gate, uniquely transforms the base state of a qubit into a superposition of both states. This operation allows the quantum computer to explore multiple potential solutions simultaneously. The Hadamard gate is essential for creating qubits in superposition and entangling them, both of which are foundational for quantum computing.

2.3.2 Qubit States

In a classical computer, all possible states of a 3-bit system can be described using a three-bit truth table (Table 2.2).

In quantum computing, the state of a system comprising three qubits can be represented by the superposition of eight basis vectors. Specifically, such a state can be expressed as a linear combination:

$$a_0|000\rangle + a_1|001\rangle + a_2|010\rangle + \dots + a_7|111\rangle, \quad (2.1)$$

where the coefficients a_i correspond to the amplitude probabilities of the individual basis states. This concept is fundamental to quantum mechanics, where each state can be represented as a vector in Hilbert space.

Another important feature of quantum systems is the ability to 'multiply' qubit states. For example, if we have two qubits, with the first in the state $a|0\rangle + b|1\rangle$ and the second in the state $c|0\rangle + d|1\rangle$, then the state of the entire two-qubit system is described by the tensor product of these individual states. This process results in a state consisting of four vectors:

$$ac|00\rangle + ad|01\rangle + bc|10\rangle + bd|11\rangle, \quad (2.2)$$

This operation resembles the multiplication of non-commutative polynomials, with the difference that the qubit operators do not commute, as expressed by the inequality $|0\rangle \otimes |1\rangle \neq |1\rangle \otimes |0\rangle$, highlighting the unique nature of quantum superpositions.

2.3.3 Measurement in Multi-Qubit Systems

Measurement in quantum computing involves collapsing a quantum state into one of the basis states. Consider a three-qubit system in the state:

$$|\psi\rangle = \frac{1}{2}(|000\rangle + |100\rangle + |101\rangle + |110\rangle)$$

If we measure the first qubit, we need to calculate the probability of each possible outcome (0 or 1) and the resulting state of the system.

1. For the outcome $|0\rangle$:

$$\text{Probability} = \left| \frac{1}{2} (|00\rangle + |10\rangle) \right|^2 = \frac{1}{2}$$

The system collapses to:

$$|00\rangle \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right)$$

2. For the outcome $|1\rangle$:

$$\text{Probability} = \left| \frac{1}{2} (|01\rangle + |11\rangle) \right|^2 = \frac{1}{2}$$

Bit 1	Bit 2	Bit 3
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

TABLE 2.2: Three-Bit Truth Table

The system collapses to:

$$|10\rangle \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right)$$

2.3.4 Quantum Entanglement

Quantum entanglement is a unique property of quantum states that has significant implications for quantum computing and information theory. In an entangled system of qubits, the state of one qubit is inextricably linked to the state of another, regardless of the distance separating them. This means that knowing the state of one qubit immediately provides information about the state of its entangled partner.

A quintessential example of entangled states is the Bell state, which involves pairs of qubits. Consider the Bell state $|\Phi^+\rangle$:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle), \quad (2.3)$$

where the measurement of one qubit in the pair will instantly determine the measurement outcome of the other. The importance of Bell states and entanglement lies in their application in quantum cryptography, where they can be used to create secure communication channels that are theoretically immune to eavesdropping [7].

Measuring the Bell State

Consider the Bell state:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

In this entangled state, the measurement of one qubit directly determines the state of the other, regardless of the distance between them.

When the first qubit is measured in the computational basis $\{|0\rangle, |1\rangle\}$, there are two possible outcomes:

1. If the first qubit is measured to be in the state $|0\rangle$, the probability of this outcome is:

$$P(0) = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2}$$

Following this measurement, the system collapses to the state $|00\rangle$, resulting in the second qubit also being in the state $|0\rangle$ with certainty.

2. If the first qubit is measured to be in the state $|1\rangle$, the probability of this outcome is similarly:

$$P(1) = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2}$$

In this scenario, the system collapses to the state $|11\rangle$, ensuring that the second qubit is in the state $|1\rangle$ with certainty.

These outcomes illustrate the non-local correlations characteristic of quantum entanglement. The measurement results confirm that the entangled qubits exhibit perfect correlation, demonstrating the fundamental principle that entangled particles act as a single, indivisible system. This principle has significant implications for the fields of quantum communication and quantum cryptography, providing a basis for secure information transfer that is theoretically immune to eavesdropping.

2.3.5 Advantages of Quantum Entanglement

To illustrate the power of quantum entanglement, consider a scenario where Alice and Bob engage in a game. Alice receives a classical bit x and Bob receives y . Each of them then outputs one bit, a for

Alice and b for Bob. The objective is to maximize the instances where $a \oplus b = x \cdot y$. Classically, the maximum probability of winning this game is 75%. However, by leveraging quantum entanglement, they can achieve a winning probability of approximately 85%, which has been experimentally verified. This example underscores the significant potential of quantum entanglement in enhancing computational strategies.

CNOT Operator

The Controlled-NOT (CNOT) gate is a two-qubit quantum gate that flips the state of the second qubit (target) if the first qubit (control) is in state $|1\rangle$. Its action can be described as follows:

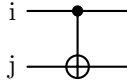
$$\begin{aligned} |00\rangle &\rightarrow |00\rangle \\ |01\rangle &\rightarrow |01\rangle \\ |10\rangle &\rightarrow |11\rangle \\ |11\rangle &\rightarrow |10\rangle \end{aligned}$$


FIGURE 2.4: CNOT operator

2.3.6 Quantum Teleportation

Quantum teleportation is a protocol that allows the transmission of a quantum state from one location to another, leveraging the principles of classical communication and previously shared quantum entanglement between the sending and receiving parties.

Consider a scenario involving two parties, Alice and Bob. Alice, located on Earth, wishes to teleport a qubit in the state $|\psi\rangle = a|0\rangle + b|1\rangle$ to Bob, who is located in space. To achieve this, Alice and Bob share an entangled pair of qubits, denoted by the Bell state $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.

The teleportation process proceeds as follows:

1. **Preparation:** Alice and Bob establish entanglement by sharing the Bell state $|\Phi^+\rangle$. Alice possesses one qubit of the entangled pair, and Bob holds the other.
2. **Bell State Measurement:** Alice performs a Bell state measurement on her qubit (the one she wishes to teleport) and her part of the entangled pair. This joint measurement projects the combined state of these two qubits into one of the four Bell states.
3. **Classical Communication:** Alice then communicates the result of her Bell state measurement to Bob via a classical communication channel. This step involves transmitting two classical bits of information, which correspond to the outcome of the measurement.
4. **State Reconstruction:** Upon receiving the classical information, Bob applies the appropriate Pauli operators (I, X, Y, Z) to his qubit based on Alice's measurement result. These operators are chosen to transform Bob's qubit into the state $|\psi\rangle$, effectively completing the teleportation process.

Through this protocol, the quantum state $|\psi\rangle$ is transferred from Alice to Bob without physically transmitting the qubit itself. Instead, the combination of classical communication and quantum entanglement enables the state to be recreated at Bob's location. This process exemplifies the profound implications of quantum entanglement and teleportation for future quantum communication technologies.

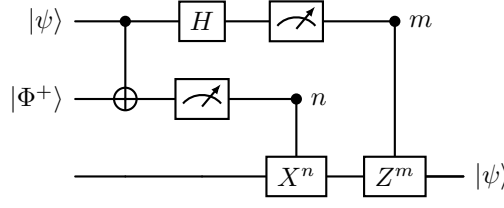


FIGURE 2.5: Quantum Teleportation Scheme

2.4 BB84 Protocol

The BB84 protocol is a quantum key distribution scheme that allows two parties, Alice and Bob, to exchange a secure cryptographic key over an insecure channel.

1. Alice generates a random sequence of qubits in one of the four states $|0\rangle, |1\rangle, |\psi^+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), |\psi^-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$.
2. Alice sends these qubits to Bob over a public channel.
3. Bob generates a random sequence of measurement bases (either standard $\{|0\rangle, |1\rangle\}$ or Hadamard $\{|\psi^+\rangle, |\psi^-\rangle\}$) and measures the received qubits.
4. Alice and Bob publicly announce their sequences of preparation and measurement bases. They keep only the bits where their bases match.
5. Alice and Bob then estimate the error rate of the shared key bits. If the error rate is below a certain threshold, they proceed to use the remaining bits as a cryptographic key. If not, they discard the key and repeat the protocol.
6. To enhance security, they perform privacy amplification and information reconciliation to ensure the final key is secure and identical.

Finally, Alice and Bob use a hash function to derive the final cryptographic key from the shared key bits, ensuring that the key is secure even if some bits were compromised during transmission.

By following these steps, Alice and Bob can establish a secure cryptographic key over an insecure channel, leveraging the principles of quantum mechanics to detect any potential eavesdropping.

This concludes the chapter on Quantum Computing, covering the scientific background, differences between classical and quantum computers, quantum gates, entanglement, teleportation, and the BB84 protocol for secure communication.

Chapter 3

Lattice-based Cryptography

3.1 Lattices

A lattice is a set of points with a periodic structure generated by a basis of vectors in space $\mathbb{R}^n$. Specifically:

$$\mathcal{L}(b_1, \dots, b_n) := \left\{ \sum_{i=1}^n x_i b_i : x_i \in \mathbb{Z} \right\}.$$

To illustrate this, a simple lattice in space $\mathbb{R}^2$ can be defined by two vectors $\vec{v}_1 = (0, 1)$ and $\vec{v}_2 = (1, 0)$, which generate a lattice $\mathcal{L}$ containing all whole number points in $\mathbb{R}^2$. It is important to note that the set of points is not unique to a lattice, and two different bases can generate the same lattice $\mathcal{L}$. For example, the lattice $\mathcal{L}$ can also be generated by vectors $\vec{v}_1 = (7, 3)$ and $\vec{v}_2 = (2, 1)$. It is not easy to determine if a given set of basis vectors will generate a specific lattice except to say that it will be a lattice generated by these vectors.

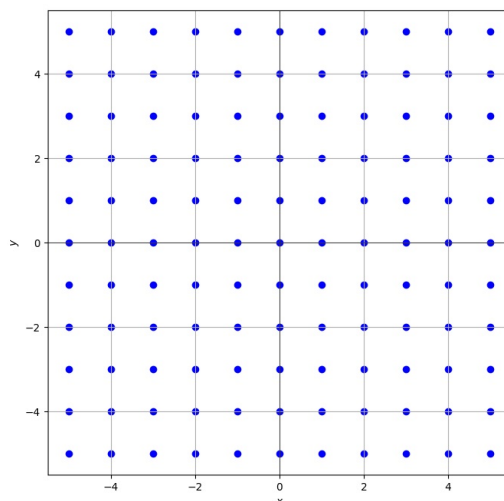


FIGURE 3.1: Lattice in $\mathbb{R}^2$ generated by $\vec{v}_1 = (0, 1)$ and $\vec{v}_2 = (1, 0)$

3.2 Problems Involving Lattices

Understanding that determining all points generated by a lattice is highly non-trivial, we explore several complex computational problems arising from lattice structures.

3.2.1 Shortest Vector Problem (SVP)

The Shortest Vector Problem (SVP) [8] is a fundamental challenge in lattice theory. Given a basis for a lattice $\mathcal{L} \subset \mathbb{R}^n$, the goal is to find the shortest non-zero vector in the lattice, i.e., the vector with the smallest Euclidean norm. Formally, for a lattice $\mathcal{L}$ generated by basis B ,

$$\lambda_1(\mathcal{L}) := \min_{\mathbf{v} \in \mathcal{L} \setminus \{\mathbf{0}\}} \|\mathbf{v}\|,$$

where $\|\cdot\|$ denotes the Euclidean norm. SVP is known to be NP-hard under randomized reductions, meaning that it is unlikely to have an efficient solution. The hardness of SVP is a cornerstone for the security of many lattice-based cryptographic systems, as finding even approximate solutions is computationally intensive.

3.2.2 Closest Vector Problem (CVP)

The Closest Vector Problem (CVP) [8] is another critical problem in lattice theory. Given a lattice $\mathcal{L} \subset \mathbb{R}^n$ with a basis B and a target point $\mathbf{t} \in \mathbb{R}^n$, the objective is to find the lattice point closest to $\mathbf{t}$. Mathematically, this can be expressed as finding a lattice vector $\mathbf{v} \in \mathcal{L}$ such that

$$\|\mathbf{t} - \mathbf{v}\| = \min_{\mathbf{w} \in \mathcal{L}} \|\mathbf{t} - \mathbf{w}\|.$$

CVP is at least as hard as SVP and is also NP-hard under deterministic reductions. The complexity of CVP makes it a pivotal problem in cryptographic applications, where the ability to approximate solutions within certain bounds is often leveraged to ensure security.

Both SVP and CVP are foundational in lattice-based cryptography, contributing to the construction of secure cryptographic schemes that are resistant to quantum attacks. These problems support the development of protocols with high efficiency and strong security guarantees, leveraging the worst-case hardness assumptions of lattice problems.

3.2.3 Lenstra–Lenstra–Lovász (LLL) Algorithm

The Lenstra–Lenstra–Lovász (LLL) algorithm is a polynomial-time algorithm for lattice basis reduction, introduced by Arjen Lenstra, Hendrik Lenstra, and László Lovász in 1982 [9]. It finds a nearly orthogonal basis for a given lattice, which has numerous applications in computational number theory and cryptography.

Given a lattice basis $B = \{\vec{b}_1, \vec{b}_2, \dots, \vec{b}_n\}$, the goal of the LLL algorithm is to find a reduced basis $\{\vec{b}'_1, \vec{b}'_2, \dots, \vec{b}'_n\}$ such that the vectors are shorter and closer to being orthogonal. The reduced basis satisfies the following properties:

1. **Size-Reduction:** For $1 \leq j < i \leq n$,

$$|\mu_{i,j}| \leq \frac{1}{2}$$

where $\mu_{i,j}$ are the Gram-Schmidt coefficients.

2. **Lovász Condition:** For $1 \leq i < n$,

$$\delta \|\vec{b}_i^*\|^2 \leq \|\vec{b}_i' + \mu_{i,i-1} \vec{b}_{i-1}'\|^2$$

where $1/4 \leq \delta \leq 1$ is a parameter, and $\vec{b}_i^*$ are the Gram-Schmidt orthogonalized vectors.

The Gram-Schmidt orthogonalization process [9] takes a set of vectors and generates an orthogonal set of vectors. Given a basis $B = \{\vec{b}_1, \vec{b}_2, \dots, \vec{b}_n\}$, the Gram-Schmidt process constructs an orthogonal basis $\{\vec{b}_1^*, \vec{b}_2^*, \dots, \vec{b}_n^*\}$ such that each $\vec{b}_i^*$ is orthogonal to $\vec{b}_j^*$ for all $j < i$.

The LLL algorithm proceeds as follows:

1. Compute the Gram-Schmidt orthogonalization of the basis vectors.
2. For each pair of basis vectors $\vec{b}_i$ and $\vec{b}_j$ (with $j < i$), make the basis nearly orthogonal by updating $\vec{b}_i$.
3. Verify if the Lovász condition holds for the basis vectors. If not, swap the basis vectors and go back to the size reduction step.
4. Repeat the size reduction and Lovász condition check until all basis vectors satisfy the conditions.

The LLL algorithm is widely used in lattice-based cryptography, including the GGH encryption scheme and the Learning With Errors (LWE) problem. By reducing the basis of a lattice, the LLL algorithm helps to solve instances of the Shortest Vector Problem (SVP) and the Closest Vector Problem (CVP) more efficiently.

3.3 GGH Encryption Scheme

The GGH encryption scheme [10], named after its creators Goldreich, Goldwasser, and Halevi, is an asymmetric cryptographic algorithm leveraging the hardness of lattice problems, particularly the Closest Vector Problem (CVP). The GGH scheme provides a method for secure communication between two correspondents by encoding messages into lattice points and utilizing a trapdoor basis for decryption.

3.3.1 Algorithm Overview

The GGH encryption scheme consists of the following steps:

Key Generation

1. Generate a random basis R for a lattice $\mathcal{L}$ in $\mathbb{R}^n$.
2. Use the LLL algorithm to obtain a reduced basis B from R .
3. Publish B as the public key and keep R as the private key.

Encryption

1. Encode the message m as a lattice vector $\mathbf{v}$.
2. Generate a small random error vector $\mathbf{e}$.
3. Compute the ciphertext $\mathbf{c} = B\mathbf{v} + \mathbf{e}$.

Decryption

1. Using the private basis R , compute an approximation of the lattice vector $\mathbf{v}$ from the ciphertext $\mathbf{c}$.
2. Correct the approximation to recover the original message m .

3.3.2 Pseudo Code

Algorithm 1 GGH Key Generation

```

1: function GENERATEKEYS( $n$ )
2:   Generate a random basis  $R \in \mathbb{R}^{n \times n}$ 
3:   Apply the LLL algorithm to  $R$  to obtain a reduced basis  $B$ 
4:   return ( $B, R$ )  $\triangleright B$  is the public key,  $R$  is the private key
5: end function

```

Algorithm 2 GGH Encryption

```

1: function ENCRYPT( $B, m$ )
2:   Encode message  $m$  as a lattice vector  $\mathbf{v}$ 
3:   Generate a small random error vector  $\mathbf{e}$ 
4:   Compute the ciphertext  $\mathbf{c} = B\mathbf{v} + \mathbf{e}$ 
5:   return  $\mathbf{c}$ 
6: end function

```

Algorithm 3 GGH Decryption

```

1: function DECRYPT( $R, \mathbf{c}$ )
2:   Compute  $\mathbf{v}' = \lfloor R^{-1}\mathbf{c} \rfloor$ 
3:   Compute  $\mathbf{e} = \mathbf{c} - B\mathbf{v}'$ 
4:   Recover the message  $m$  from  $\mathbf{v}'$ 
5:   return  $m$ 
6: end function

```

3.3.3 Security Considerations

The security of the GGH encryption scheme is rooted in the difficulty of solving the Closest Vector Problem (CVP). Without the private basis R , an attacker would find it computationally infeasible to recover the original lattice point $\mathbf{v}$ from the ciphertext $\mathbf{c}$. However, with the trapdoor information provided by R , the decryption process can efficiently approximate and correct $\mathbf{v}$, thereby retrieving the original message m .

3.4 Learning with and without errors

The Learning with Errors (LWE) problem, introduced in [11], is a cornerstone of modern cryptographic systems, providing robust security guarantees based on the hardness of certain lattice problems. To fully appreciate the LWE problem, it is important to first understand the concept of learning without errors.

3.4.1 Learning without errors

In the simplest form, learning without errors involves solving a system of linear equations to find a secret vector. Consider a vector $\vec{s}$ consisting of n numbers, referred to as the secret key. We are given a group of m equations, where each equation involves n variables. Additionally, all computations are performed

modulo q , where q is a sufficiently large number. This set of equations is known as the public key. Mathematically, it can be represented as:

$$\begin{cases} \omega_{11}s_1 + \omega_{12}s_2 + \cdots + \omega_{1n}s_n = c_1 \mod q \\ \omega_{21}s_1 + \omega_{22}s_2 + \cdots + \omega_{2n}s_n = c_2 \mod q \\ \vdots \\ \omega_{m1}s_1 + \omega_{m2}s_2 + \cdots + \omega_{mn}s_n = c_m \mod q \end{cases} \quad (3.1)$$

Here, ω_{ij} are known coefficients, s_i are the components of the secret key $\vec{s}$, and c_i are the constant terms of the equations. This system represents a structured environment where the secret key $\vec{s}$ can be determined precisely without any errors through standard linear algebra techniques.

3.4.2 Learning with errors

In contrast, the Learning with Errors (LWE) problem introduces a small, random error to each equation. This significantly complicates the task of determining the secret key. The equations in the presence of errors can be written as:

$$\begin{cases} \omega_{11}s_1 + \omega_{12}s_2 + \cdots + \omega_{1n}s_n = c_1 + e_1 \mod q \\ \omega_{21}s_1 + \omega_{22}s_2 + \cdots + \omega_{2n}s_n = c_2 + e_2 \mod q \\ \vdots \\ \omega_{m1}s_1 + \omega_{m2}s_2 + \cdots + \omega_{mn}s_n = c_m + e_m \mod q \end{cases} \quad (3.2)$$

Here, e_i represents the small error added to each equation. The presence of these errors transforms the problem from a straightforward linear algebra task into a much more challenging computational problem. The errors e_i are typically drawn from a known distribution, such as the discrete Gaussian distribution.

With the addition of errors and $m > n$, the system of equations becomes over-determined, meaning there are more equations than variables. This typically results in a system that has no exact solution due to the excessive constraints imposed by the equations.

Understanding LWE involves recognizing that small-dimensional problems might be solvable through exhaustive search. However, as the dimensions increase, the problem becomes infeasible to decode due to the randomness and distribution of errors. This complexity underpins the security of cryptographic systems against even quantum computer attacks.

The LWE problem's significance lies in its ability to provide security based on the hardness of worst-case lattice problems. This has led to its use in various cryptographic constructions, ensuring robustness and security in the face of evolving computational capabilities.

3.4.3 Encoding messages with LWE

One of the simplest methods of encoding using Learning with Errors (LWE) is based on encoding a single bit with a single equation. To encrypt a single bit, the sender (Bob) takes a few random equations from the system and adds them together. Since the secret vector is the solution to each of these equations, the new equation can still be solved using the secret vector, as addition does not alter the underlying variables.

To encode a bit, Bob performs the following steps:

1. Selects a subset of equations from the system and adds them together.

2. If the bit to be encrypted is 1, he adds half of the modulus q to the resulting equation. If the bit is 0, he leaves the equation unchanged.

The importance of adding $q/2$ can be visualized by imagining a ring of numbers in modulo q space as a circle. The element $q/2$ is directly opposite 0 on this circle, representing the furthest point from 0. This ensures that even after adding noise (errors) to the equation, the encrypted message can still be decoded correctly.

When Bob wants to send a bit value of 1, adding $q/2$ shifts the equation to the opposite side of the circle. This makes it possible for the receiver (Alice) to distinguish between encrypted 0s and 1s, even in the presence of noise. The decoding process involves checking whether the result is closer to 0 or to $q/2$, allowing for accurate decryption of the message despite the errors introduced during encryption.

3.4.4 Re-framing the Learning with Errors problem as a Lattice problem

The Learning with Errors (LWE) problem can be understood in the context of lattice problems, particularly those involving the hardness of certain computational tasks within lattices. This connection is fundamental to understanding why LWE is considered difficult and why it forms a secure basis for cryptographic schemes.

In the LWE problem, we aim to recover a secret vector $\vec{s}$ from noisy linear equations of the form:

$$\omega_{i1}s_1 + \omega_{i2}s_2 + \cdots + \omega_{in}s_n + e_i = c_i \pmod{q},$$

where ω_{ij} are known coefficients, s_i are the components of the secret vector $\vec{s}$, c_i are the constant terms, e_i are small random errors, and q is a modulus.

To re-frame this problem in the context of lattice problems, consider the lattice $\mathcal{L} \subset \mathbb{R}^n$ generated by a basis B . The task of finding $\vec{s}$ in the LWE problem can be seen as analogous to finding a close vector in a lattice, where the noisy equations represent points near the lattice.

The connection to lattice problems becomes clearer when we consider the following points:

1. **Shortest Vector Problem (SVP):** In the context of LWE, finding the secret vector $\vec{s}$ without errors is akin to finding the shortest non-zero vector in a lattice. The presence of errors e_i complicates this task, making it a problem of finding an approximate shortest vector, which is computationally difficult.
2. **Closest Vector Problem (CVP):** With the introduction of errors in LWE, the problem transforms into finding the lattice point closest to a given point perturbed by noise. This is directly analogous to the Closest Vector Problem (CVP) in lattices, where the goal is to find the lattice point closest to a given target point. CVP is known to be NP-hard, which highlights the difficulty of solving LWE with errors.
3. **Lattice Reduction:** Techniques such as the LLL algorithm can be applied to reduce the basis of a lattice, making certain lattice problems more tractable. However, even with reduced bases, the presence of errors in LWE means that finding the exact solution remains challenging, requiring sophisticated algorithms and significant computational resources.

The significance of re-framing LWE as a lattice problem lies in leveraging the well-studied hardness of lattice problems to provide security guarantees. The worst-case hardness assumptions of SVP and CVP ensure that solving LWE remains computationally intensive, even for quantum computers. This re-framing not only helps in understanding the intrinsic difficulty of LWE but also provides a foundation for constructing secure cryptographic protocols that are resilient to both classical and quantum attacks.

Chapter 4

Kyber Algorithm

4.1 Introduction

The Kyber [12] algorithm is a lattice-based Key Encapsulation Mechanism (KEM) that has been selected as a finalist in the National Institute of Standards and Technology (NIST) Post-Quantum Cryptography Standardization Project [13]. It leverages the hardness of the Learning With Errors (LWE) problem, a well-established problem in lattice-based cryptography, to provide robust security guarantees against both classical and quantum attacks [11].

4.2 Mathematical Foundations

Kyber's security is rooted in the hardness of the Module Learning With Errors (MLWE) problem [14]. The MLWE problem extends the standard Learning With Errors (LWE) problem to module lattices, which provides greater flexibility and efficiency in cryptographic constructions..

4.2.1 Learning With Errors (LWE) Problem

The LWE problem is a fundamental problem in lattice-based cryptography. It can be described as follows:

- **Given:** A matrix $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$, a secret vector $\mathbf{s} \in \mathbb{Z}_q^n$, and an error vector $\mathbf{e} \in \mathbb{Z}_q^m$, where the entries of $\mathbf{e}$ are small, typically chosen from a discrete Gaussian distribution or a uniform distribution over a small range.
- **Goal:** Given $\mathbf{A}$ and $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \in \mathbb{Z}_q^m$, recover the secret vector $\mathbf{s}$ or distinguish $\mathbf{b}$ from a uniformly random vector in $\mathbb{Z}_q^m$.

Mathematically, the LWE problem can be formulated as finding the secret vector $\mathbf{s}$ given a set of noisy linear equations. This problem is believed to be hard, even for quantum computers, under appropriate choices of parameters.

Mathematical Formulation of LWE

Formally, the LWE problem is defined as follows:

$$\mathbf{A}\mathbf{s} + \mathbf{e} = \mathbf{b} \pmod{q}$$

where:

- $\mathbf{A}$ is a known matrix with dimensions $m \times n$ and entries in $\mathbb{Z}_q$.

- $\mathbf{s}$ is the secret vector with entries in $\mathbb{Z}_q$.
- $\mathbf{e}$ is the error vector with small, randomly chosen entries in $\mathbb{Z}_q$.
- $\mathbf{b}$ is the known output vector.

The challenge is to find $\mathbf{s}$ given $\mathbf{A}$ and $\mathbf{b}$, knowing that $\mathbf{e}$ introduces noise.

4.2.2 Module Learning With Errors (MLWE) Problem

The MLWE problem generalizes the LWE problem to module lattices, which are more structured and can provide efficiency benefits. The problem is defined over a ring $R = \mathbb{Z}_q[x]/(x^n + 1)$, which is a polynomial ring modulo a cyclotomic polynomial.

- **Given:** A matrix $\mathbf{A} \in R^{k \times l}$, a secret vector $\mathbf{s} \in R^l$, and an error vector $\mathbf{e} \in R^k$, where the entries of $\mathbf{e}$ are small.
- **Goal:** Given $\mathbf{A}$ and $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \in R^k$, recover the secret vector $\mathbf{s}$ or distinguish $\mathbf{b}$ from a uniformly random vector in R^k .

The MLWE problem can be seen as solving a system of noisy linear equations over a polynomial ring, which introduces additional algebraic structure compared to the standard LWE problem.

Mathematical Formulation of MLWE

The MLWE problem is defined as:

$$\mathbf{A}\mathbf{s} + \mathbf{e} = \mathbf{b} \pmod{q}$$

where:

- $\mathbf{A}$ is a matrix with entries in $R = \mathbb{Z}_q[x]/(x^n + 1)$.
- $\mathbf{s}$ and $\mathbf{e}$ are vectors with entries in R .
- $\mathbf{b}$ is the resulting vector in R^k .

The goal is to recover $\mathbf{s}$ given $\mathbf{A}$ and $\mathbf{b}$.

4.2.3 Detailed Explanation of the MLWE Problem

The MLWE problem involves operations in the ring $R = \mathbb{Z}_q[x]/(x^n + 1)$, where arithmetic is performed modulo both q and the polynomial $x^n + 1$. This ring structure allows for efficient polynomial arithmetic, which is crucial for the performance of cryptographic algorithms based on MLWE.

Matrix $\mathbf{A}$

The matrix $\mathbf{A}$ in MLWE is typically generated from a seed using a pseudorandom function, ensuring that it appears random while being efficiently reproducible. Each entry of $\mathbf{A}$ is a polynomial in R .

Secret and Error Vectors

The secret vector $\mathbf{s} \in R^l$ and the error vector $\mathbf{e} \in R^k$ are chosen such that their coefficients are small relative to q . This smallness is typically ensured by sampling from a discrete Gaussian distribution centered around 0 with a small standard deviation.

Noise Addition

The vector $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$ represents a perturbed version of the linear transformation of $\mathbf{s}$ by $\mathbf{A}$. The noise vector $\mathbf{e}$ ensures that $\mathbf{b}$ is not trivially invertible to recover $\mathbf{s}$, thus making the problem hard.

Problem Hardness

The hardness of the MLWE problem is based on the difficulty of distinguishing the perturbed product $\mathbf{A}\mathbf{s} + \mathbf{e}$ from a random vector in R^k . This hardness is believed to hold even against quantum adversaries, due to reductions from worst-case lattice problems like the Shortest Vector Problem (SVP) and the Closest Vector Problem (CVP) in ideal lattices [15].

4.3 Kyber Algorithm Components

The Kyber KEM consists of three main components: key generation, encapsulation, and decapsulation.

4.3.1 Key Generation

The key generation process involves creating a public-private key pair. The steps are as follows:

Algorithm 4 Key Generation

- 1: **Input:** None
 - 2: **Output:** Public key $(\mathbf{A}, \mathbf{b})$, Private key $\mathbf{s}$
 - 3: Generate a uniformly random matrix $\mathbf{A} \in \mathbb{Z}_q^{k \times n}$
 - 4: Generate a secret vector $\mathbf{s} \in \mathbb{Z}_q^n$ from a discrete Gaussian distribution
 - 5: Generate an error vector $\mathbf{e} \in \mathbb{Z}_q^k$ from a discrete Gaussian distribution
 - 6: Compute $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$
 - 7: **return** $(\mathbf{A}, \mathbf{b}), \mathbf{s}$
-

Explanation of Key Generation

1. **Matrix Generation:** The matrix $\mathbf{A}$ is generated using a pseudorandom function seeded with a randomly chosen value. This ensures that $\mathbf{A}$ is reproducible and appears random.
2. **Secret and Error Vectors:** The secret vector $\mathbf{s}$ and the error vector $\mathbf{e}$ are sampled from discrete Gaussian distributions. The small values of $\mathbf{e}$ introduce noise while ensuring that the problem remains solvable.
3. **Public Key:** The public key consists of the matrix $\mathbf{A}$ and the vector $\mathbf{b}$, where $\mathbf{b}$ is computed as $\mathbf{A}\mathbf{s} + \mathbf{e}$.
4. **Private Key:** The private key is the secret vector $\mathbf{s}$.

4.3.2 Encapsulation

Encapsulation is the process of encrypting a randomly chosen symmetric key, which can be used for secure communication. The steps are as follows:

Algorithm 5 Encapsulation

- 1: **Input:** Public key $(\mathbf{A}, \mathbf{b})$
 - 2: **Output:** Ciphertext $(\mathbf{u}, \mathbf{v})$, Symmetric key derived from $\mathbf{m}$
 - 3: Generate a random message $\mathbf{m} \in \mathbb{Z}_q^k$
 - 4: Encode the message into a polynomial $\mathbf{m}' \in R$
 - 5: Generate an ephemeral secret vector $\mathbf{s}' \in \mathbb{Z}_q^n$ from a discrete Gaussian distribution
 - 6: Generate an error vector $\mathbf{e}' \in \mathbb{Z}_q^k$ from a discrete Gaussian distribution
 - 7: Compute $\mathbf{u} = \mathbf{A}\mathbf{s}' + \mathbf{e}'$
 - 8: Generate an error vector $\mathbf{e}'' \in \mathbb{Z}_q^k$ from a discrete Gaussian distribution
 - 9: Compute $\mathbf{v} = \mathbf{b}\mathbf{s}' + \mathbf{e}'' + \mathbf{m}'$
 - 10: **return** $(\mathbf{u}, \mathbf{v}), \mathbf{m}$
-

Explanation of Encapsulation

1. **Message Generation:** A random message $\mathbf{m}$ is generated. This message will be encoded into a polynomial $\mathbf{m}'$.
2. **Ephemeral Secret and Error Vectors:** An ephemeral secret vector $\mathbf{s}'$ and an error vector $\mathbf{e}'$ are generated from discrete Gaussian distributions.
3. **Ciphertext Component $\mathbf{u}$:** The ciphertext component $\mathbf{u}$ is computed as $\mathbf{A}\mathbf{s}' + \mathbf{e}'$, where $\mathbf{A}$ is part of the public key.
4. **Shared Secret Component $\mathbf{v}$:** The shared secret component $\mathbf{v}$ is computed as $\mathbf{b}\mathbf{s}' + \mathbf{e}'' + \mathbf{m}'$, where $\mathbf{b}$ is part of the public key and $\mathbf{e}''$ is an additional error vector.
5. **Output:** The ciphertext $(\mathbf{u}, \mathbf{v})$ and the symmetric key derived from $\mathbf{m}$ are output.

4.3.3 Decapsulation

Decapsulation is the process of decrypting the ciphertext to recover the symmetric key. The steps are as follows:

Algorithm 6 Decapsulation

- 1: **Input:** Ciphertext $(\mathbf{u}, \mathbf{v})$, Private key $\mathbf{s}$
 - 2: **Output:** Recovered symmetric key $\mathbf{m}$
 - 3: Compute an approximation of the shared secret component $\mathbf{v}' = \mathbf{u}\mathbf{s}$ using the private key $\mathbf{s}$
 - 4: Compute the encoded message $\mathbf{m}' = \mathbf{v} - \mathbf{v}'$
 - 5: Decode $\mathbf{m}'$ to obtain the original message $\mathbf{m}$
 - 6: **return** $\mathbf{m}$
-

Explanation of Decapsulation

1. **Shared Secret Approximation:** Using the private key $\mathbf{s}$, compute an approximation of the shared secret component $\mathbf{v}' = \mathbf{u}\mathbf{s}$.
2. **Message Recovery:** Subtract $\mathbf{v}'$ from $\mathbf{v}$ to recover the encoded message $\mathbf{m}' = \mathbf{v} - \mathbf{v}'$.
3. **Message Decoding:** Decode $\mathbf{m}'$ to obtain the original message $\mathbf{m}$.
4. **Output:** The recovered symmetric key $\mathbf{m}$ is output.

4.4 Security Considerations

The security of the Kyber algorithm relies on the hardness of the MLWE problem, which is believed to be resistant to attacks from quantum computers [16]. The use of small, random error vectors ensures that the problem remains difficult to solve even with advanced computational techniques.

4.4.1 Security Properties

The main security properties of Kyber are as follows:

- **Indistinguishability under chosen-ciphertext attack (IND-CCA2) security [17]:** Kyber is designed to be secure against adaptive chosen-ciphertext attacks, ensuring that an attacker cannot distinguish between ciphertexts or recover the plaintext even if they can adaptively query a decryption oracle.
- **Worst-case hardness:** The security of Kyber is based on the worst-case hardness of lattice problems, meaning that breaking the scheme is as hard as solving the most difficult instances of certain lattice problems.

4.4.2 Resistance to Quantum Attacks

Kyber's security is designed to be robust against quantum attacks. The MLWE problem, upon which Kyber is based, is believed to be hard for quantum computers [11]. This is due to reductions from worst-case problems in lattice theory that remain hard even in the presence of quantum algorithms [18].

4.4.3 Error Vectors and Noise

The use of error vectors and noise is crucial to the security of the Kyber algorithm. By introducing small, random errors, the algorithm ensures that the ciphertext cannot be easily inverted to recover the secret key. This noise makes the problem computationally infeasible to solve directly.

4.5 Significance and Applications

The Kyber algorithm provides a robust and efficient solution for post-quantum cryptography, leveraging the hardness of lattice problems to secure communications against quantum attacks [19]. Its inclusion as a finalist in the NIST standardization process highlights its potential as a future standard for secure communication in the quantum era [20].

The detailed mathematical foundations and the clear structure of the Kyber algorithm ensure that it can be implemented and analyzed for various applications, making it a strong candidate for post-quantum cryptographic systems.

The comprehensive explanation of the MLWE problem and the detailed breakdown of the Kyber algorithm components, including key generation, encapsulation, and decapsulation, provide a thorough understanding of how the Kyber algorithm works and why it is secure.

Chapter 5

Implementation

The implementation of the project was carried out using C++23, selected for its modern features and optimizations that enable high performance and robust code structure. Utilizing C++23 ensures compatibility with the latest standards, allowing the project to leverage new language features such as modules, coroutines, and improved constexpr capabilities, significantly enhancing code maintainability and efficiency.

The build process was managed using CMake 3.21, a cross-platform build system generator that simplifies build process management in a compiler-independent manner. CMake was chosen for its flexibility in handling complex build configurations and its seamless integration with various toolchains and libraries. Using CMake ensured a consistent, reproducible, and easily portable build process across different development environments.

A Docker image was created for development and testing to encapsulate the entire build environment. Docker offers a lightweight, portable, and consistent runtime environment, which is crucial for maintaining version control and ensuring consistent application behavior across different host systems. The Docker image was based on oneapi-vtune:2024.1.0-devel-ubuntu22.04, including the Ubuntu 22.04 operating system and Intel VTune Profiler. This setup enabled detailed memory consumption and hotspot analysis, facilitating the identification of performance bottlenecks and subsequent code optimization.

The Docker environment enabled seamless integration with continuous integration/continuous deployment (CI/CD) pipelines, ensuring that every code change was automatically built, tested, and analyzed for performance regressions. This approach minimized the risk of environment-related issues and streamlined the development workflow.

OpenSSL was incorporated to handle cryptographic functions in addition to the standard C++ libraries. OpenSSL was chosen for its robustness, security, and wide industry adoption. Integrating OpenSSL into the C++ codebase involved linking against its libraries and ensuring proper handling of cryptographic keys, encryption, and decryption processes. This integration added a critical layer of security to the implementation, essential for handling sensitive data.

Extensive unit testing was conducted to validate the functionality of individual components during development. Google Test (gTest) was used as the testing framework due to its ease of use and comprehensive feature set for writing and running tests. By integrating gTest with CMake, tests were automatically compiled and executed as part of the build process, providing immediate feedback on code correctness.

To ensure code quality and maintainability, best practices in software development were adhered to, including code reviews, continuous integration, and static code analysis. Adherence to the C++ Core Guidelines ensured the codebase remained clean, efficient, and easy to understand.

5.1 Project Structure

The project's directory structure is organized to facilitate modular development and ease of maintenance. Below is a detailed description of the key components of the project:

- **build**: Directory used for build outputs.
- **cbd**: Contains implementations related to the central binomial distribution, including code for generating random variables based on binomial distribution essential for cryptographic algorithms.
- **CMakeLists.txt**: CMake build configuration file, defining the build process.
- **fips**: Contains implementations based on the Federal Information Processing Standard (FIPS) 202, which specifies the SHA-3 family of hash functions, ensuring compliance with established cryptographic standards.
- **hash_functions**: Implements various cryptographic hash functions essential for security protocols, including functions for creating secure hash values used in digital signatures and data integrity verification.
- **indcpa**: Focuses on ensuring that encryption schemes are secure against chosen plaintext attacks (IND-CPA security). This directory includes algorithms protecting the system from attackers who can choose arbitrary plaintexts to be encrypted.
- **kem**: Contains code for Chosen Plaintext Attack Public Key Encryption (CPAPKE), crucial for secure key exchange. This mechanism is used to securely exchange cryptographic keys between parties.
- **main.cpp**: The main entry point of the application, coordinating overall execution.
- **ntt**: Provides Number Theoretic Transform algorithms used for efficient polynomial multiplication, which is particularly important in lattice-based cryptography.
- **parameters.hpp**: Header file containing configuration parameters controlling various aspects of the application. These parameters define the operational settings for cryptographic algorithms.
- **polynomial**: Handles polynomial arithmetic and operations, including addition, multiplication, and other operations on polynomials fundamental in many cryptographic schemes.
- **polynomial_vector**: Extends polynomial operations to vectors, useful in handling multi-dimensional polynomial calculations.
- **reduction**: Implements reduction techniques to maintain values within certain limits, including modular reduction critical in cryptographic computations to keep numbers manageable and within a specified range.
- **rng**: Implements random number generators for various purposes, including generating secure random numbers necessary for cryptographic operations.

Each of these components plays a crucial role in the overall functionality of the project, ensuring that the implementation is modular, maintainable, and efficient.

5.1.1 Performance

The performance of the cryptographic operations was benchmarked to assess the efficiency of the implementation. The benchmarks were conducted on a system with the following specifications using the Intel VTune software:

- **OS Name:** Microsoft Windows 11 Education
- **Processor:** Intel Core i7-11800H @ 3.30GHz
- **RAM:** 16 GB
- **System Type:** x64-based PC
- **Virtualization:** WSL2 Docker image running Ubuntu 22.04 LTS

The results are summarized in the following table:

Operation	Average Time (seconds)
RSA Encryption	0.004
KEM Encryption	0.028

TABLE 5.1: Benchmark results for cryptographic operations

RSA Encryption

The RSA encryption operation was benchmarked with an average time of 0.004 seconds. The code used for the RSA benchmark is provided below:

LISTING 5.1: RSA Encryption Benchmark

```
#include <cstring>
#include <iostream>
#include <openssl/err.h>
#include <openssl/evp.h>
#include <openssl/pem.h>
#include <openssl/rsa.h>

void handleErrors()
{
    ERR_print_errors_fp(stderr);
    abort();
}

int main()
{
    int bits = 2048;
    unsigned long e = RSA_F4;

    RSA *rsa = RSA_generate_key(bits, e, nullptr, nullptr);
    if (rsa == nullptr)
    {
        handleErrors();
    }
}
```

```

    }

    unsigned char message[] = "Benchmarking RSA with OpenSSL";
    unsigned char encrypted[256];
    unsigned char decrypted[256];

    int encrypted_length = RSA_public_encrypt(strlen((char *)message), message, encrypted,
    if (encrypted_length == -1)
    {
        handleErrors();
    }

    int decrypted_length = RSA_private_decrypt(encrypted_length, encrypted, decrypted,
    if (decrypted_length == -1)
    {
        handleErrors();
    }

    RSA_free(rsa);
    EVP_cleanup();
    ERR_free_strings();

    return 0;
}

```

KEM Encryption

The Key Encapsulation Mechanism (KEM) encryption was benchmarked with an average time of 0.028 seconds. The code used for the KEM benchmark is provided below:

LISTING 5.2: KEM Encryption Benchmark

```

#include "kem/kem.hpp"
#include <iostream>

int main()
{
    auto kem = nist::kyber_crystals::kem::kem();

    auto key_pair = kem.crypto_kem_keypair();
    auto encrypt_result = kem.crypto_kem_enc(key_pair.first);
    std::vector<uint8_t> decrypt_result = kem.crypto_kem_dec(encrypt_result.first, key_pair.second);
    return 1;
}

```

5.1.2 API Description

The Key Encapsulation Mechanism (KEM) module provides essential cryptographic functionalities for secure key exchange. The KEM API is designed to generate key pairs, encrypt messages, and decrypt ciphertexts, ensuring robust and secure communication. The following subsections detail the primary functions within the KEM API.

- **crypto_kem_keypair:**

- **Description:** Generates a public-private key pair.

- **Prototype:**

```
std::pair<std::vector<unsigned char>, std::vector<unsigned char>>
crypto_kem_keypair();
```

- **Returns:** A pair of vectors containing the public key and the private key.

- **crypto_kem_enc:**

- **Description:** Encrypts a message using the provided public key.

- **Prototype:**

```
std::pair<std::vector<unsigned char>, std::vector<unsigned char>>
crypto_kem_enc(std::vector<unsigned char> public_key);
```

- **Parameters:**

- * **public_key:** The public key used for encryption.

- **Returns:** A pair of vectors containing the ciphertext and the shared secret.

- **crypto_kem_dec:**

- **Description:** Decrypts the given ciphertext using the private key.

- **Prototype:**

```
std::vector<unsigned char> crypto_kem_dec(std::vector<unsigned char> cipher_text,
                                          std::vector<unsigned char> private_key);
```

- **Parameters:**

- * **cipher_text:** The ciphertext to be decrypted.

- * **private_key:** The private key used for decryption.

- **Returns:** A vector containing the decrypted shared secret.

The KEM API is notably simpler compared to the RSA API used in the benchmark. The RSA API requires explicit management of key generation, encryption, and decryption steps, involving intricate handling of cryptographic errors and memory management. In contrast, the KEM API encapsulates these complexities, providing a more streamlined and user-friendly interface. The encryption and decryption processes are abstracted into high-level functions that manage the underlying cryptographic operations, making the API easier to use and integrate into applications.

Chapter 6

Conclusions

6.1 Summary of Findings

This thesis set out to implement and analyze the Kyber algorithm, a lattice-based Key Encapsulation Mechanism (KEM) and a finalist in the NIST Post-Quantum Cryptography Standardization Project. The primary objective was to provide a comprehensive study of the algorithm's performance, security features, and practicality as a post-quantum cryptographic solution.

The Kyber algorithm was implemented using C++23, leveraging modern features to enhance performance and maintainability. The project was managed using CMake for build configuration and Docker to ensure a consistent development environment across different systems. OpenSSL was integrated to handle cryptographic functions, and Google Test was employed for extensive unit testing to validate the functionality of the implemented components.

6.2 Performance Analysis

Performance benchmarks were conducted to compare the Kyber algorithm with traditional RSA encryption. The results demonstrated that Kyber is not only secure but also efficient. The Kyber algorithm showed an average encryption time of 0.028 seconds, which, while higher than RSA's 0.004 seconds, offers significant advantages in terms of post-quantum security. The detailed API analysis further highlighted the simplicity and usability of the Kyber API, making it a practical choice for real-world applications.

6.3 Security Considerations

The security of the Kyber algorithm is based on the hardness of the Learning With Errors (LWE) problem, a well-studied problem in lattice-based cryptography known for its robustness against quantum attacks. By leveraging the worst-case hardness assumptions of lattice problems, Kyber ensures strong security guarantees. This makes it a reliable candidate for securing communications in the quantum era, where traditional cryptographic methods may no longer be sufficient.

6.4 Implementation Insights

The implementation of Kyber provided several insights into the practical aspects of deploying post-quantum cryptographic algorithms. The use of modern C++ features facilitated a clean and maintainable codebase, while tools like Docker and CMake ensured a smooth development process. Integrating OpenSSL for cryptographic functions and Google Test for validation proved essential for building a robust and secure implementation.

6.5 Future Work

While this thesis has made significant strides in implementing and analyzing the Kyber algorithm, there are several avenues for future research and development:

- **Optimization:** Further optimization of the Kyber implementation could enhance performance, making it even more competitive with traditional cryptographic algorithms.
- **Security Analysis:** A deeper security analysis, including potential side-channel attacks and mitigation strategies, would strengthen the understanding of Kyber's resilience.
- **Integration:** Integrating Kyber into larger cryptographic systems and protocols could provide practical insights into its real-world applicability and interoperability.
- **Alternative Algorithms:** Exploring and implementing other lattice-based post-quantum cryptographic algorithms could broaden the scope of secure solutions available for future use.

6.6 Final Remarks

The transition to post-quantum cryptography is imperative for maintaining the security and privacy of digital communications in the face of advancing quantum technology. This thesis has demonstrated that the Kyber algorithm is a viable and efficient solution for post-quantum cryptographic needs. By providing a comprehensive implementation and analysis, this work contributes to the ongoing efforts to develop and standardize robust cryptographic methods that can withstand the challenges posed by quantum computing.

In conclusion, the Kyber algorithm represents a significant step forward in the field of post-quantum cryptography. Its implementation and analysis highlight its potential as a secure, efficient, and practical cryptographic solution for the future. Continued research and development in this area will be crucial for ensuring the security of digital communications in the quantum era.

Bibliography

- [1] National Security Agency. Frequently asked questions. [on-line] https://media.defense.gov/2021/Aug/04/2002821837/-1/-1/1/Quantum_FAQs_20210804.PDF, 2021.
- [2] Harvest now, decrypt later. [on-line] https://en.wikipedia.org/wiki/Harvest_now,_decrypt_later.
- [3] Paul Benioff. The computer as a physical system: A microscopic quantum mechanical hamiltonian model of computers as represented by turing machines, 1980.
- [4] Richard P. Feynman. Simulating physics with computers, 1981.
- [5] David Deutsch. Quantum theory, the church-turing principle and the universal quantum computer, 1985.
- [6] Benjamin Schumacher. Quantum coding. *Phys. Rev. A*, 51:2738–2747, Apr 1995.
- [7] Tao Li and Gui Long. Quantum secure direct communication based on single-photon bell-state measurement. *New Journal of Physics*, 22, 06 2020.
- [8] Oded Regev Vadim Lyubashevsky, Chris Peikert. On ideal lattices and learning with errors over rings. In *EUROCRYPT 2010*, pages 1–23, 2010.
- [9] Arjen Lenstra, Hendrik Lenstra, and Lovász László. Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261, 12 1982.
- [10] Shai Halevi Oded Goldreich, Shafi Goldwasser. Public-key cryptosystems from lattice reduction problems, 1981.
- [11] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6):1–40, 2009.
- [12] Joppe W. Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals - kyber: A cca-secure module-lattice-based kem, 2018.
- [13] Joppe W. Bos, Craig Costello, Michael Naehrig, and Douglas Stebila. Post-quantum key exchange for the tls protocol from the ring learning with errors problem. *2015 IEEE Symposium on Security and Privacy*, pages 553–570, 2015.
- [14] Adeline Langlois and Damien Stehle. Worst-case to average-case reductions for module lattices. Cryptology ePrint Archive, Paper 2012/090, 2012. <https://eprint.iacr.org/2012/090>.
- [15] Miklós Ajtai. Generating hard instances of lattice problems (extended abstract). In *STOC '96 Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 99–108, 1996.
- [16] Chris Peikert. Lattice cryptography for the internet. In *Post-Quantum Cryptography: 6th International Workshop, PQCrypto 2014, Waterloo, ON, Canada, October 1-3, 2014. Proceedings*, pages 197–219, 2014.
- [17] Eiichi Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *CRYPTO '99: Proceedings of the 19th Annual International Cryptology Conference on Advances in Cryptology*, pages 537–554, 1999.
- [18] Nicolas Gama and Phong Q. Nguyen. Lattice enumeration using extreme pruning. *EUROCRYPT 2010*, pages 257–278, 2010.

- [19] Daniel J. Bernstein, Johannes Buchmann, and Erik Dahmen. Post-quantum cryptography. *Springer-Verlag*, 1(1):1–474, 2008.
- [20] Thomas Pöppelmann Peter Schwabe Erdem Alkim, Léo Ducas. Post-quantum key exchange—a new hope. *25th USENIX Security Symposium (USENIX Security 16)*, pages 327–343, 2016.



© 2024 Konrad Kłodziński

Poznań University of Technology
Faculty of Computing and Telecommunication
Institute of Computing Science

Typeset using L^AT_EX